Group 21

Trilingual Multimodal Customer Support Ticket Management System
Software Architecture Document

Version 1.0

Document identifier:	G21-SAD-1.0
Prepared by:	Group 21
Submission date:	30 July 2026

Revision History

Date	Version	Description	Author
30/Jul/26	1.0	Initial Software Architecture Document aligned with the submitted template and current project scope.	Group 21

Document status. Baseline architecture for implementation, model integration, testing, deployment and evaluation. Architectural decisions marked as "proposed" may be refined after prototype measurements without changing the document's principal views.

Diagram convention. All diagrams are vector diagrams produced directly in LaTeX using PGF/TikZ. Elements use black text, white backgrounds and numbered captions. Internal services are shown explicitly rather than treating the system as a black box.

Project boundary. The system accepts textual tickets and optional supporting images. Audio input and speech transcription are outside the approved project scope.

Table of Contents

Software Architecture Document

1. Introduction

This Software Architecture Document (SAD) specifies the structural and behavioural architecture of the Trilingual Multimodal Customer Support Ticket Management System. The system automates key stages of customer-support ticket handling for English, Sinhala, Tamil, Romanised Sinhala (Singlish), Romanised Tamil (Tanglish), and code-mixed inputs. It combines multilingual machine learning, image understanding and a grounded response-generation workflow while preserving human control for low-confidence or high-risk decisions.

1.1 Purpose

The document captures the significant architectural decisions, system views, interfaces, data structures, deployment topology, performance targets and quality controls required to design and implement the system. The intended audience is the project team, supervisor, evaluators, developers, data scientists, testers and future maintainers. It provides a common reference for:

- implementing the web application, API, machine-learning services and persistence layer;
- training and evaluating the multilingual and multimodal models;
- validating security, privacy, reliability and maintainability constraints; and
- tracing use cases to logical components, processes and deployment nodes.

1.2 Scope

The architecture covers ticket submission, optional image upload, multilingual preprocessing, intent classification, urgency prediction, sentiment analysis, multimodal evidence fusion, context-aware response drafting, agent review, assignment, escalation, tracking, feedback, audit logging, knowledge-base administration and model-version management.

The training baseline uses the Banking77 intent dataset and project-produced translations into Sinhala, Tamil, Singlish and Tanglish. The original English records and four translated variants form the multilingual training corpus; mixed-language examples are added through controlled augmentation and manually reviewed samples. The architecture deliberately excludes voice/audio processing, payment execution, direct access to core banking transaction systems and fully autonomous handling of critical tickets.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SAD	Software Architecture Document.
Banking77	Intent-classification dataset containing 77 online-banking intent categories.
Intent	The customer issue category predicted from a ticket, such as card payment, transfer, cash withdrawal or account access.
Singlish / Tanglish	Romanised Sinhala / Romanised Tamil written with Latin characters.
Code-mixed	One ticket containing spans from more than one language or script.
XLM-R	XLM-RoBERTa, a multilingual Transformer encoder used as the primary text representation candidate.
OCR	Optical Character Recognition used to extract visible text from screenshots or photographs.
VLM	Vision-language model used to encode image content and align visual evidence with text.
RAG	Retrieval-Augmented Generation: generation conditioned on retrieved approved knowledge.
LLM	Large Language Model used to draft a response under policy and retrieval constraints.
P1–P4	Priority levels: P1 Critical, P2 High, P3 Medium, P4 Low.
Human-in-the-loop	Mandatory human review for low-confidence, critical, sensitive or policy-restricted tickets.

PII	Personally Identifiable Information, including account numbers, identity data and contact details.
API	Application Programming Interface.
RBAC	Role-Based Access Control.

1.4 References

Primary research, standards and technology references are listed in Section 12. The diagrams were created using PGF/TikZ inside this LaTeX source; no rasterised UML tool output is required.

1.5 Overview

Section 2 defines the architectural representations. Section 3 records goals and constraints. Sections 4–9 describe the use-case, logical, process, deployment, implementation and data views. Sections 10 and 11 specify size, performance and quality characteristics. Section 12 provides IEEE-style references.

2. Architectural Representation

The architecture is represented using complementary views derived from the Rational Unified Process 4+1 model:

- **Use-Case View:** actors, architecturally significant use cases and detailed scenarios.
- **Logical View:** domain classes, service responsibilities and package decomposition.
- **Process View:** runtime workflows, concurrency boundaries and service interactions.
- **Deployment View:** physical/container nodes, networks and external dependencies.
- **Implementation View:** source-code layers, components and package dependencies.
- **Data View:** persistent entities, relationships, retention and model artefacts.

The proposed implementation starts as a **modular monolith for transactional business functions** plus independently deployable ML inference workers. This minimises integration overhead for a student project while preserving clear service boundaries that can later be separated when load or team ownership requires it. Communication is synchronous REST/JSON for user-facing operations and asynchronous messaging for OCR, batch inference, notifications, retraining and analytics.

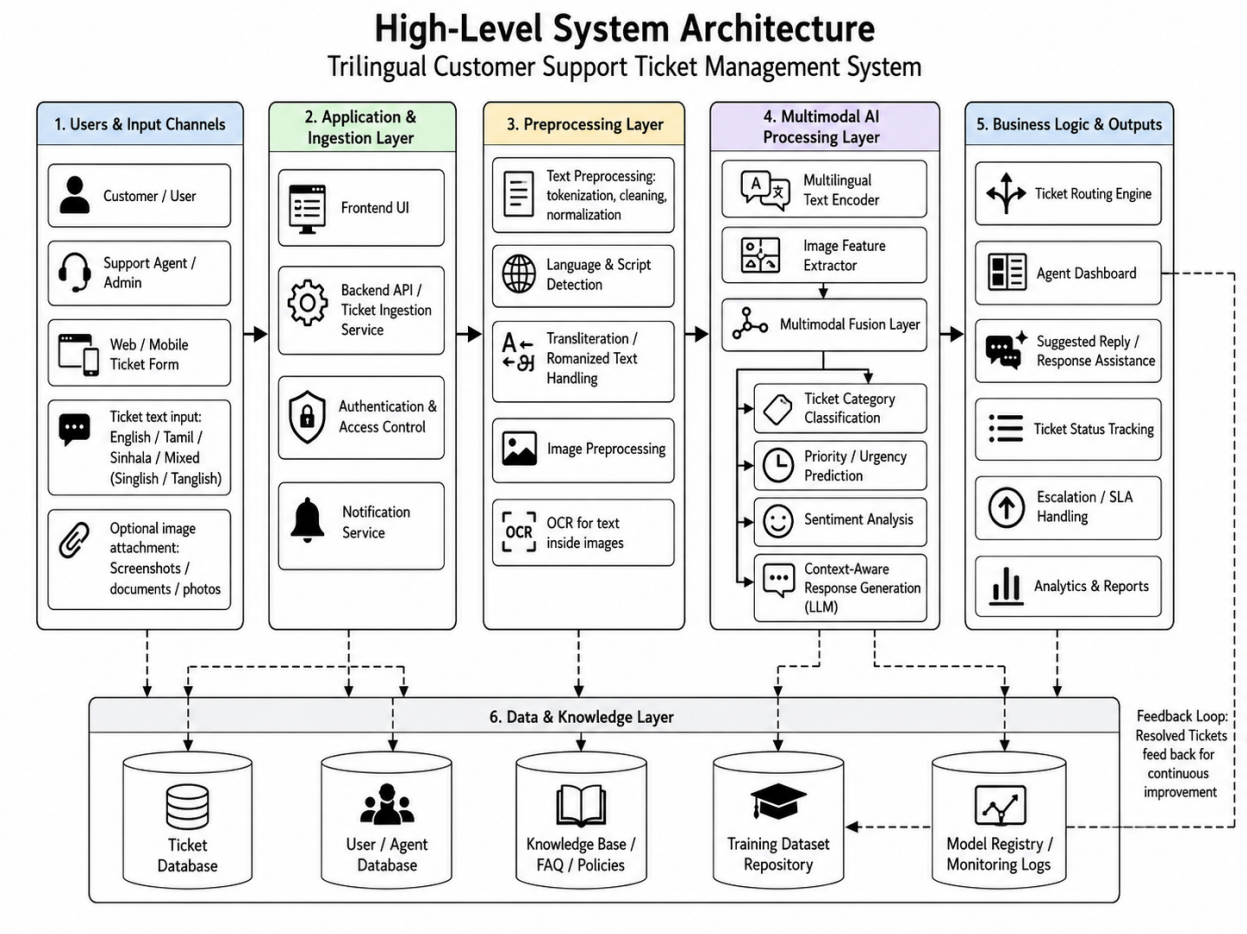

Figure 1. High-level architectural representation

Figure 1 separates transactional ticket management from model inference and response generation. The API layer coordinates these components and writes all predictions, confidence scores, model versions and user actions to auditable storage.

3. Architectural Goals and Constraints

Architectural Goals

- **Multilingual robustness:** preserve Sinhala, Tamil, English, Singlish, Tanglish and code-mixed text without forcing translation to English at runtime.
- **Multimodal evidence:** use screenshot text and visual context when an image is supplied, but remain fully functional for text-only tickets.
- **Explainable automation:** expose intent, priority, sentiment, confidence, extracted evidence and model version to agents.
- **Safe response generation:** ground drafts in approved knowledge and route uncertain or sensitive cases to human review.
- **Reproducible ML lifecycle:** version datasets, preprocessing, models, metrics and deployment artefacts.
- **Maintainable delivery:** use layered modules, stable interfaces, containerisation, automated tests and central observability.

Key Constraints

- Banking77 provides intent labels but not project-ready urgency, sentiment, image or response labels; those require annotation rules, auxiliary datasets or controlled weak supervision.
- Sinhala and Tamil are lower-resource languages than English; translated samples may contain translation artefacts and must be quality checked.
- Romanised text has inconsistent spelling; normalisation must not erase intent-bearing banking terms, account identifiers or named entities.
- Optional images may contain sensitive banking data. Files require type/size validation, encryption, access control, malware scanning and retention rules.
- Response generation must not invent account status, balances, transaction outcomes or policy. It may draft guidance but cannot execute banking operations.
- Academic-resource limits favour parameter-efficient fine-tuning, cached embeddings, mixed precision and a deployable baseline before larger-model experimentation.

Significant Architectural Decisions

ID	Decision	Rationale and consequence
ADR-01	Shared multilingual encoder	Fine-tune one XLM-R-based encoder across all text variants, with language/script metadata as auxiliary features. This supports transfer and code-mixed inputs while avoiding five isolated models.
ADR-02	Multi-task prediction heads	Intent, priority and sentiment share the encoder but use separate heads and weighted losses. This reduces duplicated inference while permitting task-specific calibration.
ADR-03	Optional gated fusion	OCR text and visual embeddings are fused only when a valid image is present. A modality mask prevents missing images from degrading text-only performance.
ADR-04	RAG with review gates	Response drafts retrieve approved articles and include provenance. Low-confidence, P1/P2 and sensitive cases require an agent.
ADR-05	Modular monolith + ML workers	Business modules remain in one deployable API initially; compute-heavy inference and OCR run behind stable interfaces and can scale independently.
ADR-06	Immutable audit trail	Predictions, overrides, response edits, status changes and model versions are append-only audit events for traceability.

4. Use-Case View

Architecturally significant actors are the Customer, Support Agent, Administrator, ML Engineer/Data Scientist and external Notification Provider. The use cases cover the complete lifecycle from intake to resolution and model improvement.

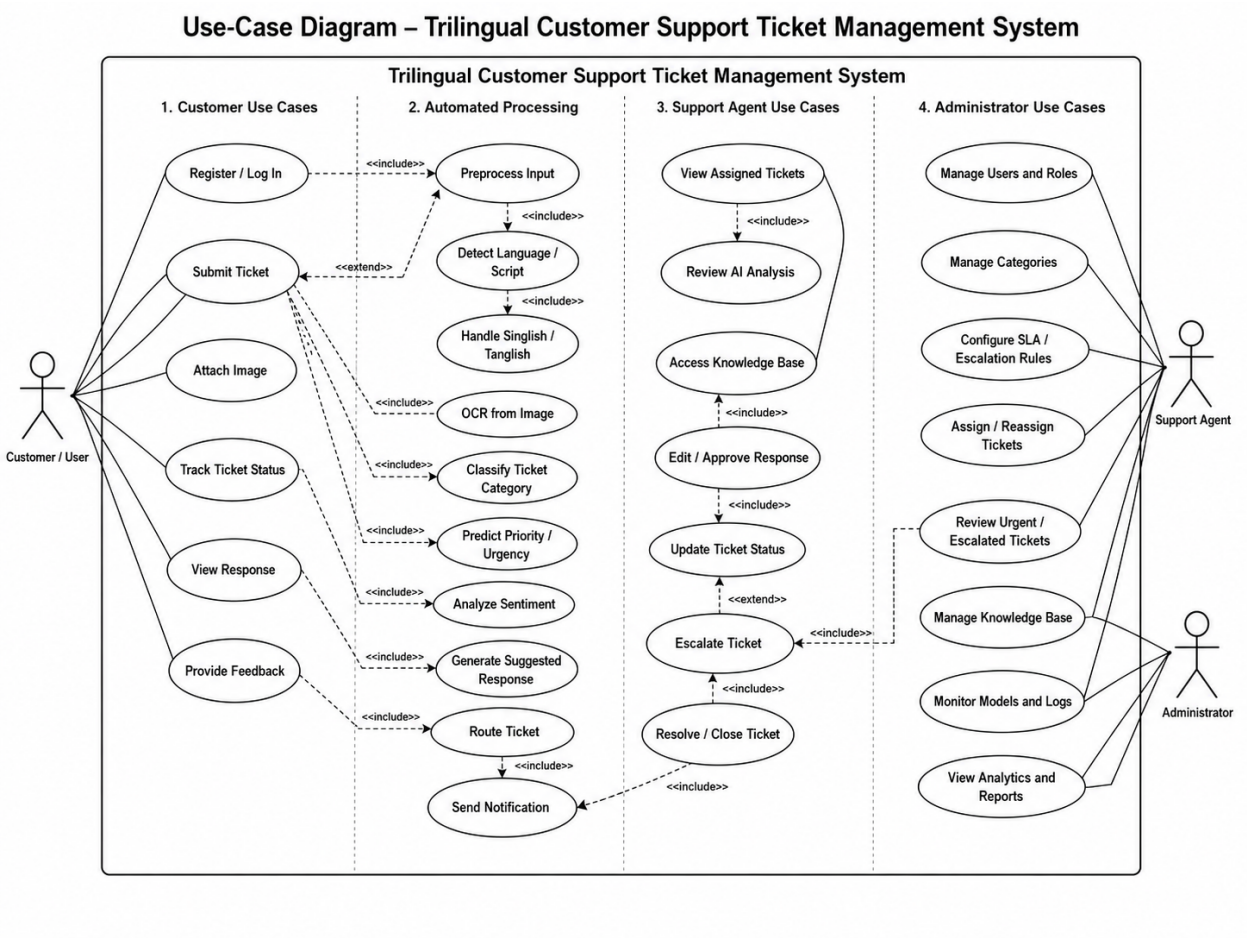

Figure 2. System use-case diagram

Figure 2 shows more than five central use cases and the internal automation they exercise. Ticket analysis and grounded response drafting are included within submission, while image evidence extends analysis only when present.

4.1 Use-Case Realizations

Use case name	**UC-01 Submit Multilingual Ticket**
Actor	Customer
Description	Create a support ticket using English, Sinhala, Tamil, Singlish, Tanglish or a mixed form; optionally attach an image.
Preconditions	Customer can access the submission interface; accepted privacy notice; file is within configured type and size limits.
Main flow	1. Customer enters subject and description. 2. UI validates mandatory fields. 3. Customer optionally attaches an image. 4. API creates a pending ticket and attachment record. 5. System queues analysis. 6. Customer receives a ticket identifier.
Successful end/post condition	Ticket is persisted, analysis is queued, and its status is visible.
Fail end/post condition	No incomplete ticket is committed; the customer receives a specific validation or service error.
Extensions	Unsupported/corrupt image is rejected; duplicate-submission warning is shown; suspected PII is masked in logs.

Use case name	**UC-02 Analyse and Classify Ticket**
Actor	System (triggered by Customer or Support Agent)
Description	Predict intent, urgency and sentiment from text and optional image evidence.
Preconditions	Ticket is stored; at least one valid text field exists; an active model version is available.
Main flow	1. Normalise Unicode and mask sensitive patterns. 2. Detect script/language spans for metadata. 3. Tokenise raw/code-mixed text. 4. For an image, run validation, OCR and visual encoding. 5. Fuse available modality representations. 6. Produce intent, priority and sentiment probabilities. 7. Calibrate confidence and persist prediction with model version.
Successful end/post condition	Ticket has auditable predictions, confidence scores and extracted evidence.
Fail end/post condition	Ticket is marked "analysis failed" and routed for manual triage without data loss.
Extensions	Low confidence triggers manual review; P1 prediction triggers immediate escalation; absent image uses text-only pathway.

Use case name	**UC-03 Generate Context-Aware Response Draft**
Actor	System, Support Agent
Description	Create a safe draft grounded in approved knowledge and ticket context.
Preconditions	Ticket analysis is available; knowledge index is online; response policy permits drafting.
Main flow	1. Build retrieval query from ticket, intent and evidence. 2. Retrieve approved articles. 3. Construct prompt with language preference, policy and citations. 4. LLM generates a draft. 5. Guardrails check prohibited claims, PII leakage and missing grounding. 6. Persist draft and provenance.
Successful end/post condition	A reviewable response draft and source references are attached to the ticket.
Fail end/post condition	No ungrounded text is sent; the ticket is queued for manual drafting.
Extensions	For P1/P2 or low-confidence tickets, generation may occur but sending is disabled until agent approval.

Use case name	**UC-04 Review, Send and Resolve Ticket**
Actor	Support Agent
Description	Inspect predictions and evidence, edit the draft, send the final response and update status.
Preconditions	Agent is authenticated and authorised; ticket is assigned or accessible to the agent.
Main flow	1. Agent opens ticket. 2. UI displays text, image, OCR, predictions and confidence. 3. Agent accepts or corrects labels. 4. Agent edits/approves response. 5. Notification service sends it. 6. Ticket status becomes resolved or awaiting customer. 7. Actions are audited.
Successful end/post condition	Customer receives the response; ticket history and feedback are persisted.
Fail end/post condition	Status remains unchanged and the failed send is retryable.
Extensions	Agent escalates, requests more information or marks the ticket as duplicate/spam.

Use case name	**UC-05 Assign or Escalate Ticket**
Actor	Support Agent, Administrator
Description	Route a ticket to an appropriate queue or specialist according to intent, priority and policy.
Preconditions	Ticket exists; actor has routing permission; target queue/user is active.
Main flow	1. System recommends queue and SLA. 2. Actor confirms or changes recommendation. 3. Assignment is written transactionally. 4. Notification is queued. 5. SLA timer and audit event are updated.
Successful end/post condition	Exactly one current assignment exists and escalation history is retained.
Fail end/post condition	Previous assignment remains valid; no partial routing state is stored.
Extensions	Automatic P1 escalation; unavailable queue falls back to general support.

Use case name	**UC-06 Manage Knowledge and Model Versions**
Actor	Administrator, ML Engineer
Description	Curate approved response knowledge and promote validated model versions.
Preconditions	Actor has elevated permission; candidate artefacts pass integrity checks.
Main flow	1. Admin creates/updates knowledge article. 2. Article passes review and becomes active. 3. ML engineer registers model, dataset and metrics. 4. Candidate runs validation. 5. Authorised actor promotes version. 6. Services reload through versioned configuration.
Successful end/post condition	Active knowledge/model version is uniquely identifiable and rollback-ready.
Fail end/post condition	Existing production version remains active.
Extensions	Shadow deployment and A/B evaluation; emergency rollback; article expiry.

5. Logical View

5.1 Overview

The logical design separates presentation, application orchestration, domain rules, ML inference, external integration and persistence. Domain objects contain business invariants; application services coordinate transactions; ML adapters expose versioned prediction contracts; infrastructure modules implement databases, object storage, messaging and external APIs.

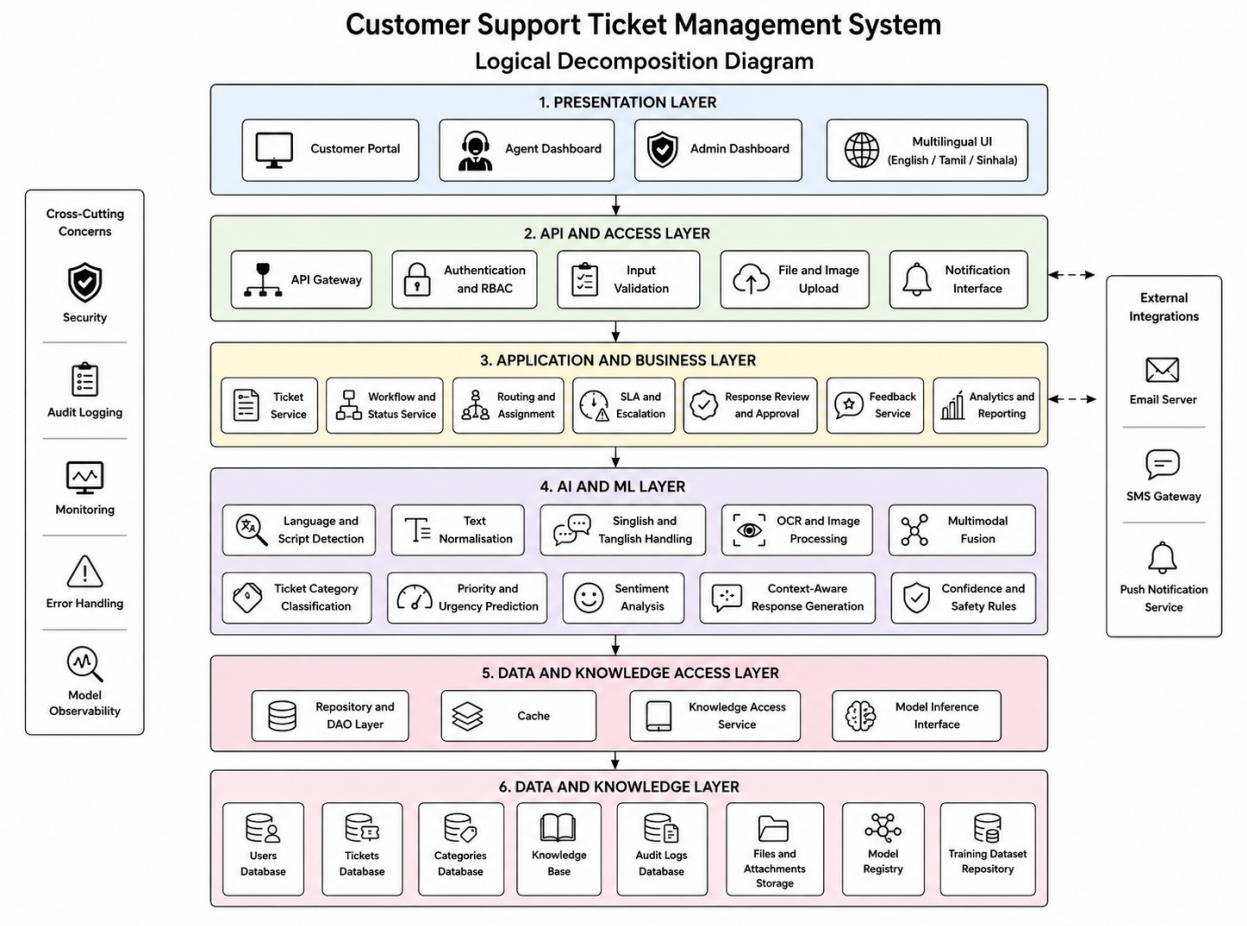

Figure 3. Logical layer decomposition

5.2 Architecturally Significant Design Packages

Package	Responsibilities	Significant classes/components
presentation	Submission form, agent dashboard, administration screens, localisation and client-side validation.	TicketForm, TicketWorkspace, QueueDashboard, ModelAdminView
application	Use-case orchestration, transactions, status transitions, policy checks and event publication.	TicketApplicationService, AnalysisOrchestrator, AssignmentService, ResponseWorkflow
domain	Business entities and invariants independent of frameworks.	Ticket, Assignment, Priority, Prediction, ResponseDraft, KnowledgeArticle
ml.text	Text cleaning, language/script metadata, tokenisation, encoder inference and calibration.	TextPreprocessor, LanguageProfiler, MultilingualEncoder, IntentHead
ml.vision	Image validation, OCR, visual embeddings and modality quality scoring.	ImageValidator, OCRService, VisualEncoder, EvidenceExtractor
ml.fusion	Optional multimodal fusion and multi-task output aggregation.	FusionModel, ModalityMask, ConfidenceCalibrator
generation	Retrieval, prompt construction, LLM gateway, safety and provenance.	Retriever, PromptBuilder, LLMGateway, ResponseGuardrail
persistence	Repository interfaces, database mappings, object storage and migrations.	TicketRepository, ModelRegistryRepository, AuditRepository
integration	Email/SMS/webhook adapters and monitoring exporters.	NotificationGateway, MalwareScanner, MetricsExporter

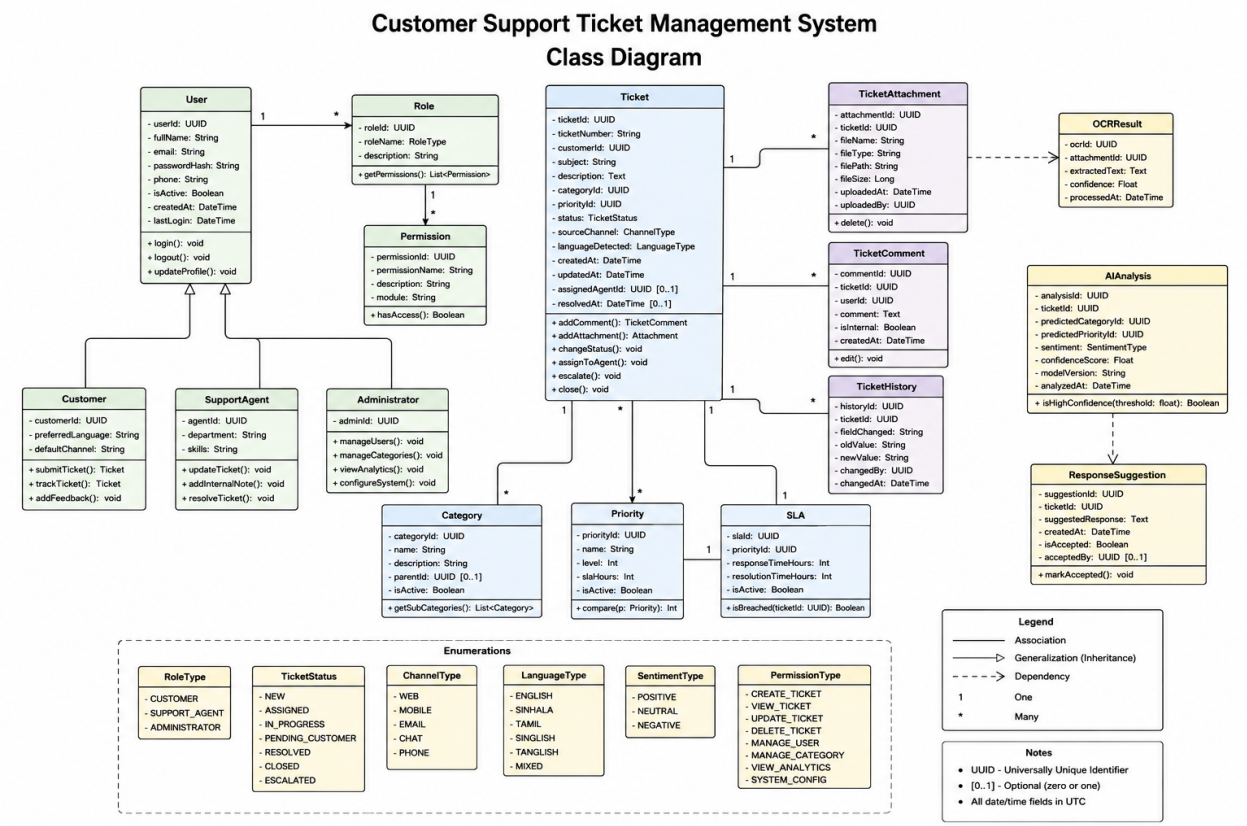

Figure 4. Architecturally significant class diagram

Figure 4 emphasises persistence and traceability. A ticket may have multiple attachments, predictions and response drafts; every prediction references the model version that produced it. Assignments and audit events preserve the operational history rather than overwriting it.

6. Process View

The runtime process model uses synchronous request-response for interactive actions and asynchronous jobs for compute-intensive or retryable work. A queue decouples ticket intake from OCR and model inference. Workers are stateless; durable state is held in PostgreSQL, object storage and the model/knowledge registries.

Main Runtime Processes

- **Web/API process:** validates requests, enforces RBAC, controls transactions and exposes status.
- **Analysis worker:** preprocesses text, invokes text/image models, fuses evidence and writes predictions.
- **Generation worker:** retrieves approved knowledge, invokes the LLM and applies response guardrails.
- **Notification worker:** sends customer/agent notifications with retry and idempotency keys.
- **Training pipeline:** validates datasets, trains candidates, calculates per-language metrics and registers artefacts.

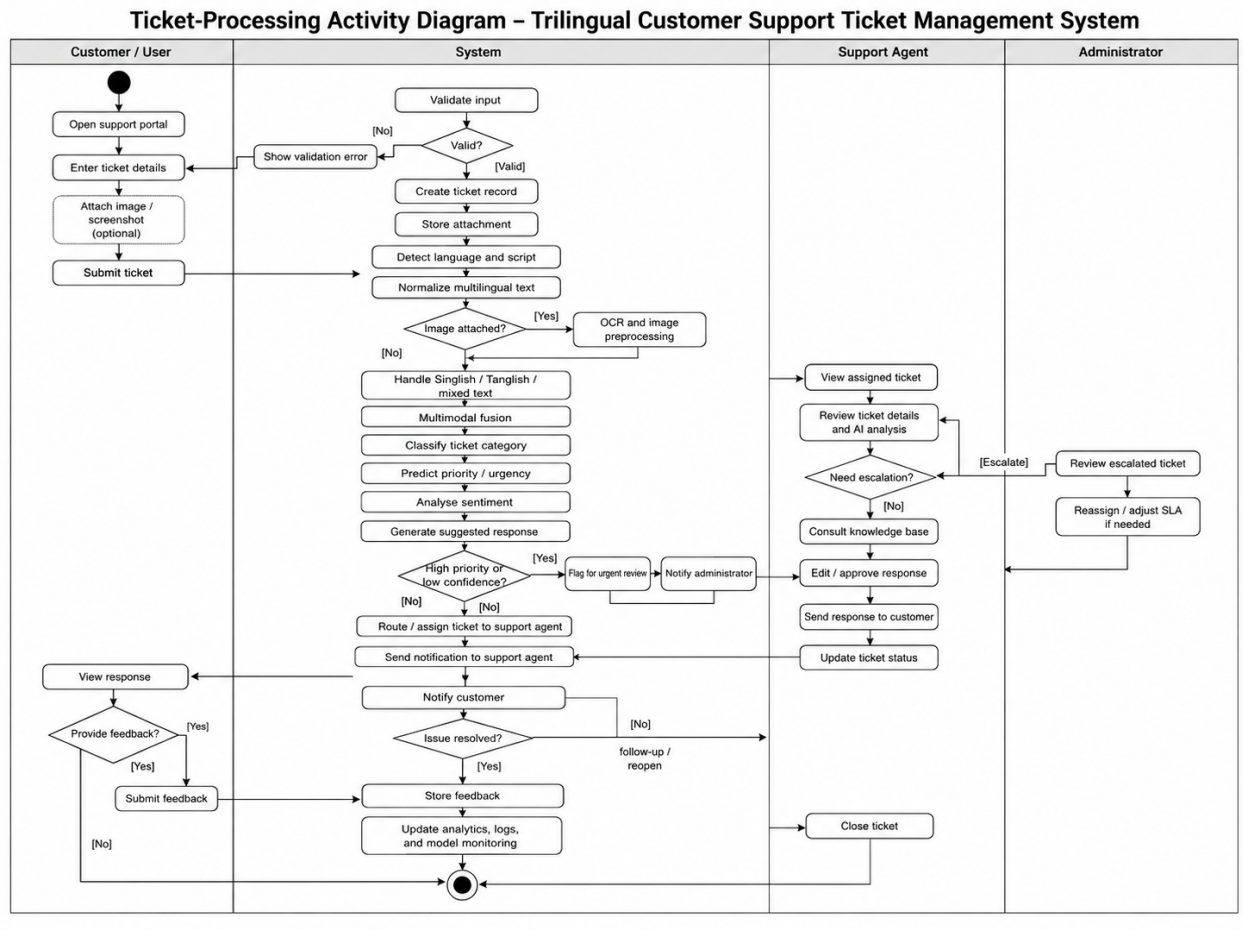

Figure 5. Ticket-processing activity diagram

Figure 5 shows that the image path is optional and that confidence/policy checks occur before response handling. Critical or uncertain tickets converge on an agent-review path.

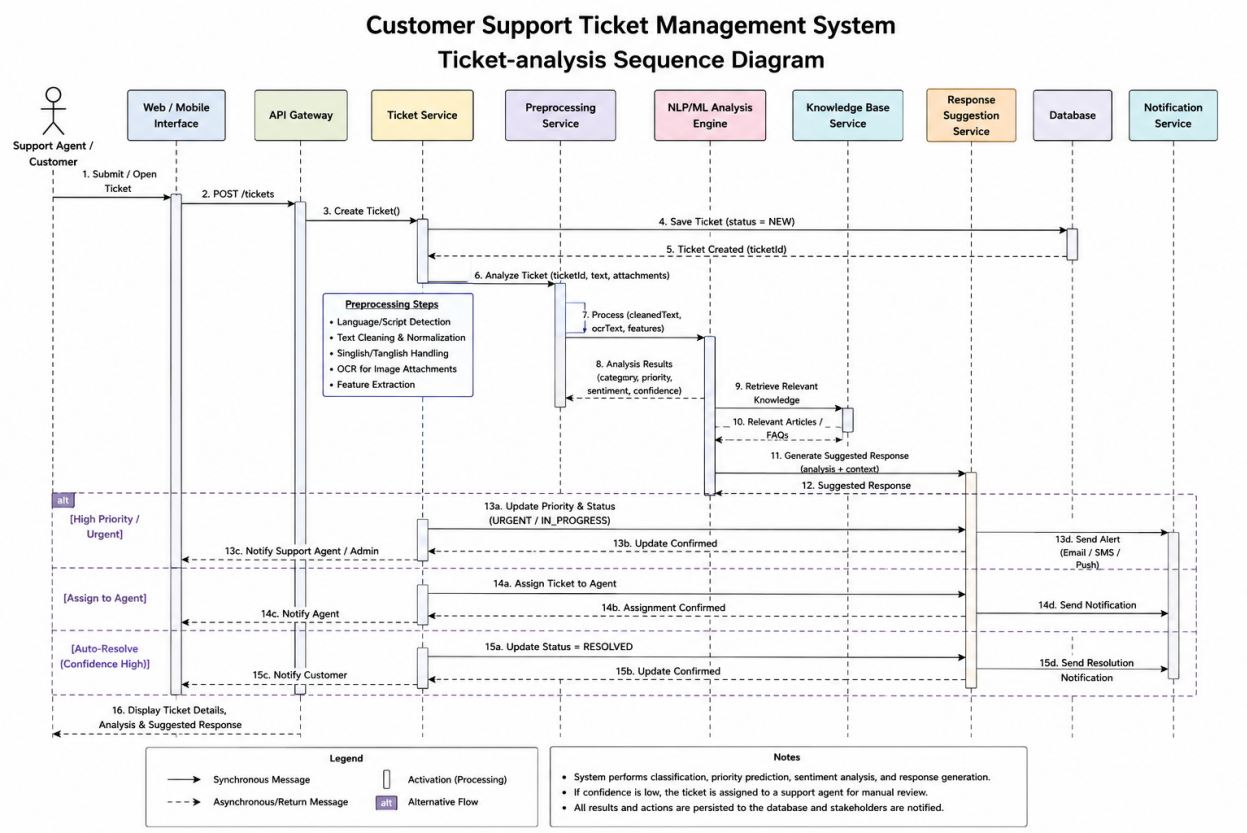

Figure 6. Sequence diagram for submission, analysis and response drafting

Figure 6 includes the internal controller, repositories, preprocessing, separate modality services, fusion, retrieval/generation and notification components. Database writes occur before user acknowledgement and before side-effecting notifications, which are retried asynchronously.

Communication and Concurrency

REST/JSON is used between UI and API and may also be used for separately deployed inference services. Message-queue events contain identifiers rather than raw sensitive text where possible. Consumers use idempotency keys so retries do not create duplicate predictions, responses or notifications. Model instances are loaded once per worker process and shared across concurrent requests subject to GPU/CPU memory limits.

7. Deployment View

The minimum demonstrator can run through Docker Compose on one workstation. The target deployment separates public access, application services, compute workers and data services across protected network zones. All external traffic uses TLS; databases and object storage are not directly internet-accessible.

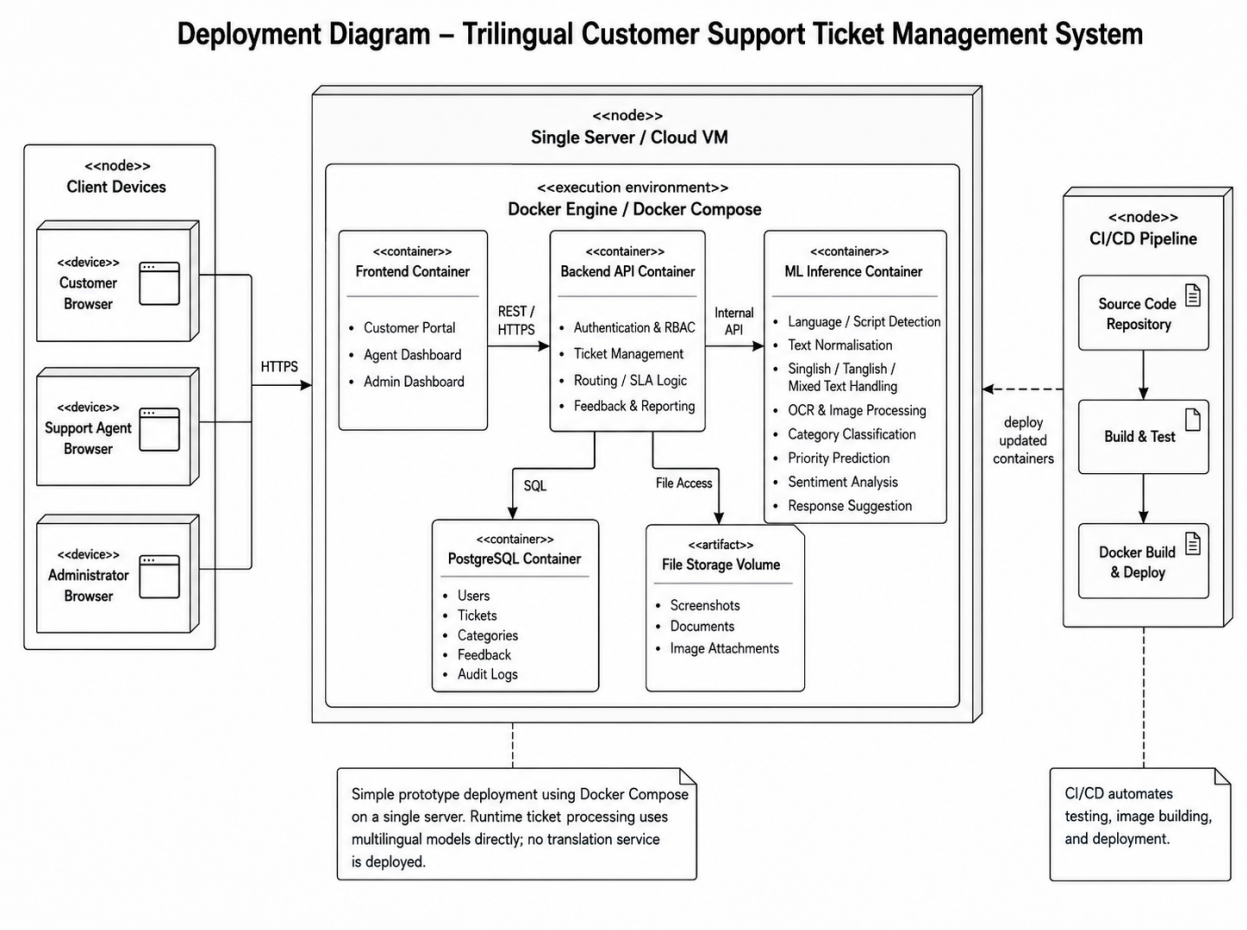

Figure 7. Deployment diagram

Figure 7 maps the API process to an application node and compute-intensive processes to worker nodes. PostgreSQL stores transactions and metadata, object storage stores images and model artefacts, and a vector index stores retrievable knowledge embeddings. For the academic prototype, these nodes may be containers on one host; the interfaces remain identical when scaled out.

Deployment Configuration

Node	Deployed artefacts	Protection / scaling
Client	Responsive web application and static assets.	Browser sandbox; no model secrets.
Reverse proxy	TLS termination, request-size limits, rate limiting and routing.	Public subnet; horizontal replication.
Application	FastAPI-compatible API, RBAC, workflow and repositories.	Private subnet; stateless replicas.
ML worker	XLM-R encoder, task heads, calibrators and fusion model.	GPU optional; replicas by queue depth.
Vision worker	Malware scan, OCR and visual encoder.	Sandboxed processing; strict file limits.
Generation worker	Retrieval, prompt policy, LLM adapter and guardrails.	Egress allow-list; key vault for credentials.
Data tier	PostgreSQL, object storage, vector index, queue/cache.	Encryption, backups, least privilege.
Observability	Central logs, metrics, traces and alerts.	Restricted operator access; PII redaction.

8. Implementation View

8.1 Overview

The implementation model follows dependency inversion: presentation and infrastructure depend on application-defined interfaces, while domain classes remain independent of web, database and model frameworks. Model-specific code is hidden behind versioned adapters so an encoder, OCR engine or LLM provider can be replaced without changing ticket workflows.

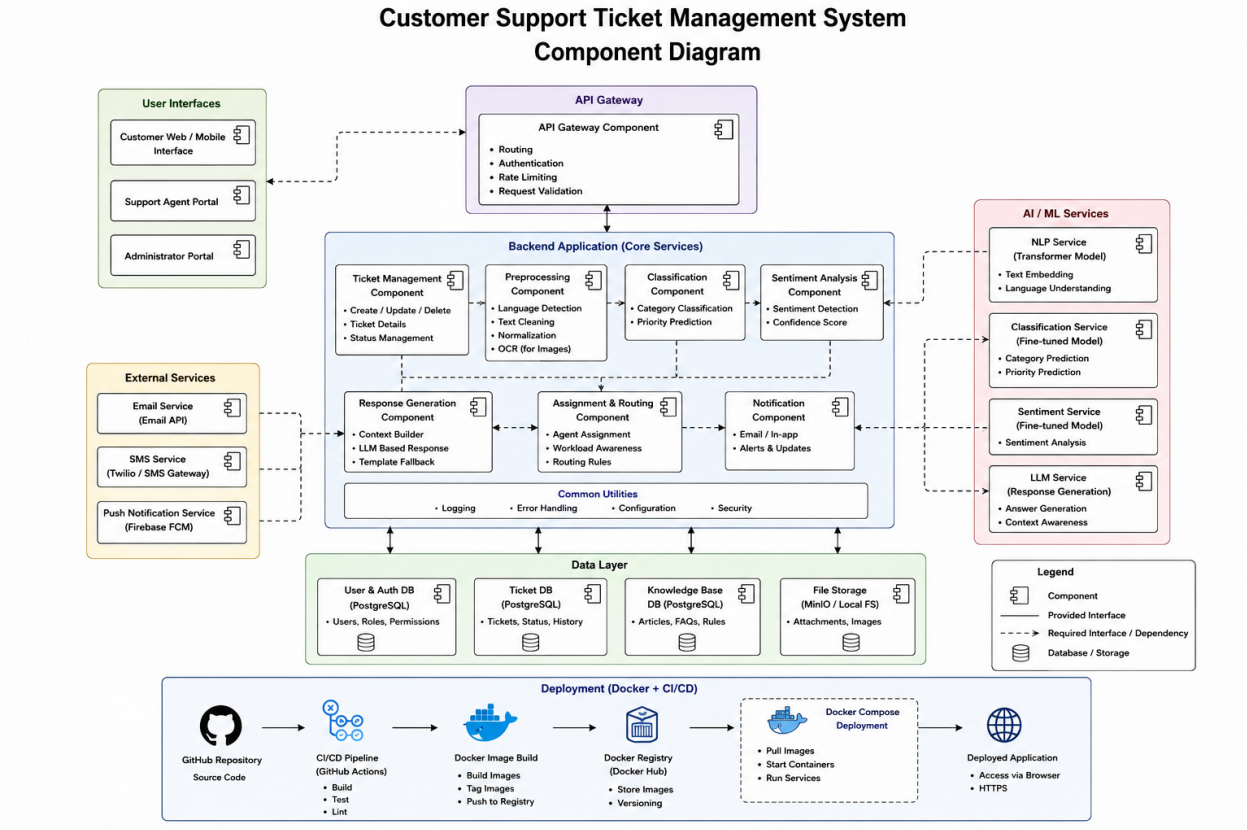

Figure 8. Implementation component diagram

Figure 8 identifies stable component boundaries. The Analysis Orchestrator does not embed model code; it invokes Text Inference, Vision/OCR and Fusion components through typed contracts. Response Generation is isolated from direct database access except through retrieval and repository adapters.

8.2 Layers

Presentation Layer

Contains the customer submission form, ticket status view, agent queue, ticket workspace, administration screens, localisation resources and API client. It performs usability validation but does not enforce domain security by itself.

API and Application Layer

Contains request schemas, authentication middleware, controllers, use-case services, transaction boundaries, workflow state machines, policy evaluation and event publication. This layer is the sole entry point for interactive clients.

Domain Layer

Contains Ticket, Prediction, Assignment, ResponseDraft, KnowledgeArticle, ModelVersion and AuditEvent entities; value objects; status-transition rules; SLA calculations; and interfaces for repositories and external gateways.

Machine-Learning Layer

Contains data schemas, preprocessing pipelines, dataset validation, tokenizer wrappers, multilingual encoder, task heads, OCR adapter, visual encoder, modality fusion, calibration, evaluation and model registry integration.

Integration and Persistence Layer

Implements PostgreSQL repositories, object-storage access, vector retrieval, message queue, notification gateways, hosted/local LLM adapters, secrets, logging, metrics and tracing.

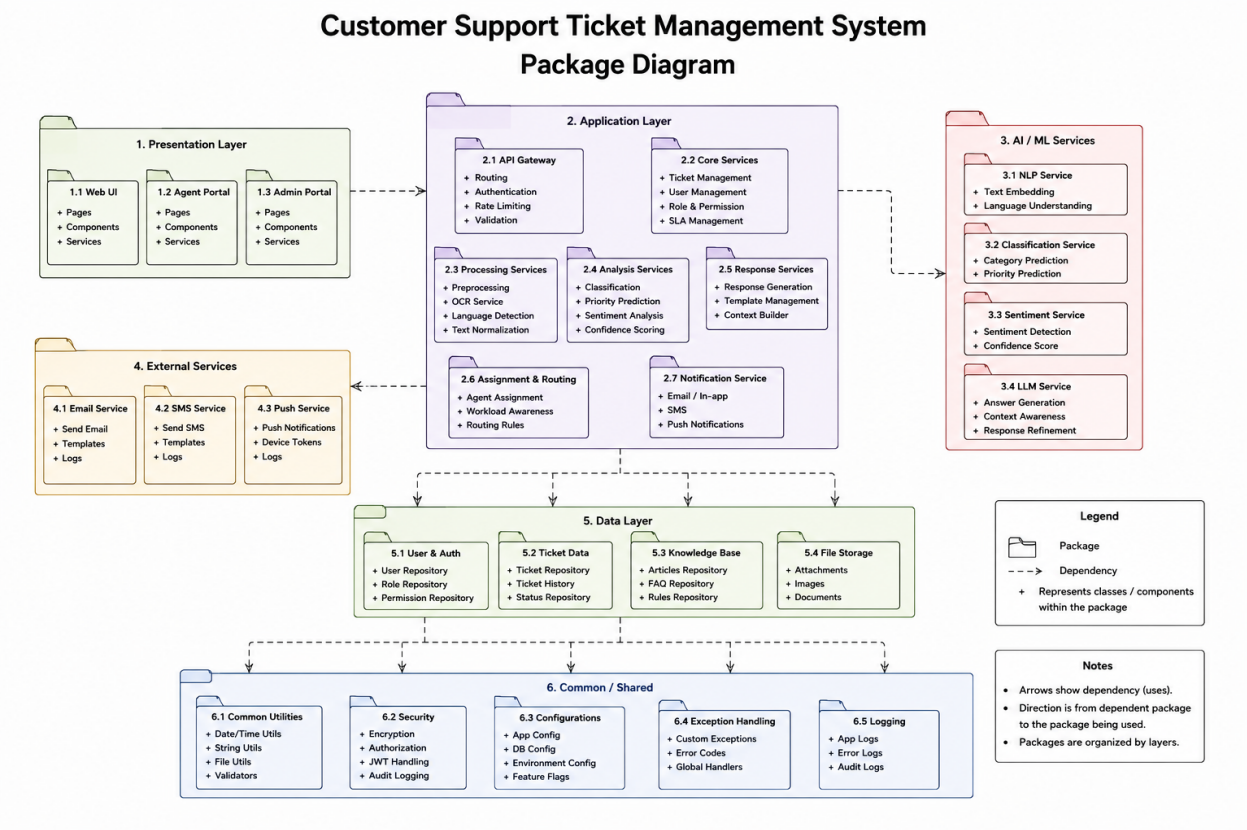

Figure 9. Package diagram and dependency direction

Figure 9 shows that domain interfaces are central. Persistence and integration implement those interfaces; tests target domain, application and ML packages. Circular dependencies between UI, application and infrastructure are prohibited.

Proposed Repository Structure

```
src/
  ui/                # customer and agent web interfaces
  api/               # routes, schemas, authentication
  application/       # use cases, workflow, orchestration
  domain/            # entities, policies, repository ports
  ml/
    data/             # Banking77 translations and validation
    preprocessing/    # native, romanised and mixed-text handling
    text/             # multilingual encoder and task heads
    vision/           # OCR and visual encoder
    fusion/           # modality fusion and calibration
    evaluation/       # per-language and cross-modal metrics
  generation/        # retrieval, prompts, LLM and guardrails
  persistence/       # database and object-store adapters
  integration/       # notifications and external gateways
```

```
  infrastructure/     # configuration, queue, logging, monitoring
tests/                # unit, integration, model and end-to-end tests
models/               # version metadata; large artefacts stored externally
```

9. Data View (optional)

Persistent data is essential because the system must retain ticket history, predictions, response provenance, model versions and feedback. Transactional records are stored in PostgreSQL; uploaded images and large model artefacts are stored in object storage; approved knowledge embeddings are stored in a vector index with source-document identifiers.

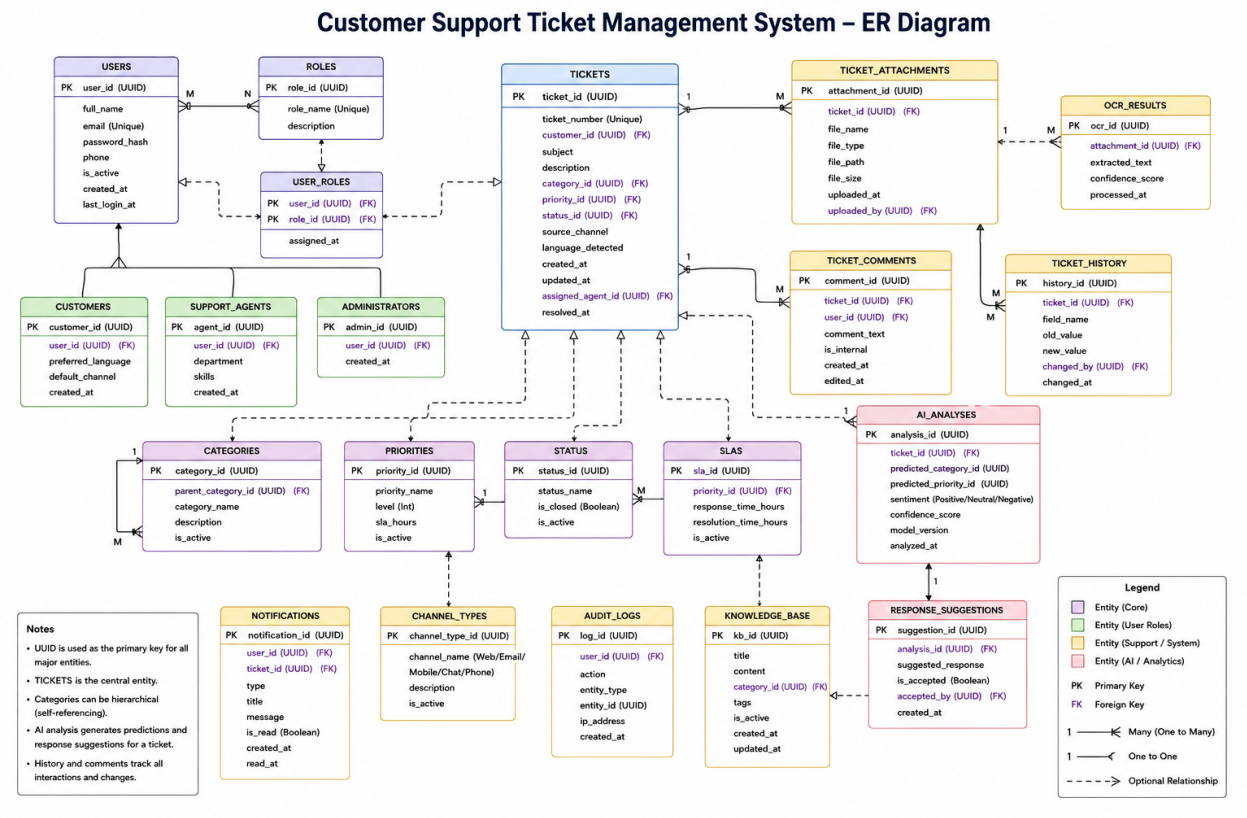

Figure 10. Persistent data model

Data Integrity and Lifecycle

- Ticket status transitions and assignment changes occur in database transactions with optimistic locking.
- Raw images are private objects referenced by opaque identifiers; public URLs are never stored.
- Predictions are immutable records. Corrections create feedback records rather than modifying historical model output.
- Knowledge articles are versioned; a response stores the exact article versions used during generation.
- Training data is stored separately from production tickets. Production reuse for training requires de-identification and explicit governance approval.
- Retention periods are configurable. Deletion removes or anonymises PII while preserving non-identifying audit evidence required for evaluation.

Training and Feature Data

The base multilingual dataset is derived from 13,083 Banking77 examples across 77 intents. Assuming complete one-to-one translation coverage, English plus Sinhala, Tamil, Singlish and Tanglish yields approximately 65,415 text instances before deduplication, quality filtering and code-mixed augmentation. Dataset manifests record source row, language variant, translator/method, review state, split, checksum and label. Splits are grouped by source row so translated versions of the same English ticket cannot leak across train and test sets.

10. Size and Performance

Dimensioning Assumptions

Characteristic	Initial target	Architectural implication
Intent classes	77	Single classification head with calibrated probabilities and top-k output.
Language variants	5 primary variants plus mixed inputs	Shared encoder; grouped data split; per-language metrics.
Text length	512 subword tokens maximum	Truncation policy preserves subject and most recent/important spans.
Image upload	Up to 10 MB; JPEG/PNG/PDF screenshot policy configurable	Streaming upload, content sniffing, object storage and asynchronous OCR.
Concurrent interactive users	50 for demonstrator	Stateless API; connection pooling; cached reference data.
Ticket ingestion	20 tickets/second burst target	Queue buffering and horizontally scalable workers.
Data retention	Configurable, default project test data only	Partitioned audit tables and object lifecycle rules.

Latency and Throughput Targets

- Ticket creation API: p95 below 800 ms excluding file-upload transfer time.
- Text-only prediction: p95 below 3 s on the target inference environment.
- Ticket with image: p95 below 8 s for OCR, visual encoding and fused prediction.
- Grounded response draft: p95 below 12 s when the configured LLM is available.
- Agent dashboard queries: p95 below 2 s for normal filtered queue sizes.
- Asynchronous notification: first attempt within 30 s after a committed workflow event.

Performance is measured end-to-end and per component. The system records queue delay, preprocessing time, model time, retrieval time, generation time, database time and external-provider time. Batch inference, quantisation, ONNX/runtime optimisation or GPU use may be introduced only after a correct baseline and profiling evidence.

Model Performance Targets

The first acceptance baseline targets macro-F1 and accuracy reported overall and separately for English, Sinhala, Tamil, Singlish, Tanglish and mixed inputs. Intent classification should exceed a simple TF–IDF/logistic-regression baseline on every reviewed language split. Priority and sentiment targets are set after label-quality analysis. Calibration is assessed using expected calibration error and reliability diagrams; low-confidence thresholds are chosen to optimise safe coverage rather than accuracy alone. Response drafts are evaluated for groundedness, policy compliance, language appropriateness, completeness and human edit distance.

11. Quality

The architecture applies ISO/IEC 25010 quality characteristics as follows.

Characteristic	Architectural contribution	Verification evidence
Functional suitability	Explicit use cases, multi-task outputs, optional image path and controlled response workflow.	End-to-end acceptance tests and traceability matrix.
Performance efficiency	Stateless API, queued workers, model caching, batch inference and database indexes.	Load tests, p50/p95/p99 metrics and resource profiles.
Compatibility	Versioned REST/JSON contracts and replaceable model/notification adapters.	Contract tests and schema compatibility checks.
Usability	Multilingual input, clear confidence/evidence display, accessible agent workflow and actionable errors.	User testing and accessibility review.
Reliability	Durable queue, idempotent consumers, transaction boundaries, retries, health checks and rollback.	Failure-injection tests, backup restore and recovery drills.
Security	TLS, RBAC, least privilege, secret management, file scanning, encryption and audit trails.	Threat model, dependency scanning and access-control tests.
Maintainability	Layering, dependency inversion, model registry, configuration and automated tests.	Static analysis, coverage, architecture checks and review.
Portability	Containerised services, environment-based configuration and externalised storage.	Docker Compose run and deployment on a second host.
Safety / AI quality	Confidence gates, human review, provenance, guardrails and immutable prediction history.	Red-team cases, hallucination checks and override analysis.

Security and Privacy Controls

- Authenticate users and enforce RBAC for customer, agent, administrator and ML-engineer operations.
- Mask account/card-like sequences in logs and prompts; prohibit raw sensitive fields in analytics events.
- Validate MIME type and file signature, limit size and dimensions, scan attachments and isolate OCR processing.
- Encrypt data in transit and at rest; rotate secrets; use short-lived signed object access.
- Record security-relevant actions and retain model/prompt/source versions needed for incident review.
- Never use a generated response as evidence of a transaction outcome; request authoritative verification through approved channels.

Reliability and Recovery

The API commits ticket state before publishing asynchronous work through an outbox pattern or equivalent transactionally safe mechanism. Workers retry transient failures with bounded exponential backoff and send exhausted jobs to a dead-letter queue. Database backups, object-versioning and model artefact checksums support recovery. A production model can be rolled back without schema changes because predictions reference version identifiers rather than model-specific columns.

Test Strategy

- **Unit tests:** domain transitions, normalisation rules, romanised-text handling, priority policy and prompt guardrails.
- **Data tests:** schema, label validity, duplicate/leakage detection, translation coverage and class balance.

- **Model tests:** reproducibility, per-language metrics, robustness to code mixing, calibration and missing-image behaviour.
- **Integration tests:** API–database, queue–worker, object storage, retrieval and notification adapters.
- **Security tests:** RBAC, injection, malicious uploads, PII logging, rate limits and secret exposure.
- **End-to-end tests:** submission through response/assignment with traceable audit events.

Risks and Mitigations

Risk	Mitigation
Translated data preserves English phrasing and inflates reported cross-language performance.	Group split by source row; native-speaker review; collect naturally authored test tickets; report translated and natural-test results separately.
Inconsistent Singlish/Tanglish spelling.	Character/subword encoder, conservative normalisation, augmentation and error analysis by pattern.
Mixed-language tickets absent from base data.	Controlled span mixing, real pilot samples, language-span metadata and dedicated mixed test set.
Image OCR is unreliable for small or blurred screenshots.	Quality score, preprocessing, OCR confidence threshold and text-only fallback; show extracted evidence to agent.
LLM hallucination or policy violation.	Retrieval grounding, restricted prompt, source display, deterministic guardrails and mandatory review gates.
Class imbalance and similar Banking77 intents.	Stratified/grouped splits, macro-F1, class weights, confusion analysis and hierarchical error review.
Compute limitations.	Parameter-efficient fine-tuning, mixed precision, caching, quantisation and smaller baseline models.

12. References

[1] I. Casanueva, T. Temčinas, D. Gerz, M. Henderson, and I. Vulić, "Efficient Intent Detection with Dual Sentence Encoders," arXiv:2003.04807, 2020. Available: `https://arxiv.org/abs/2003.04807`. Accessed: 30-Jul-2026.

[2] A. Conneau et al., "Unsupervised Cross-lingual Representation Learning at Scale," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 8440–8451, doi: 10.18653/v1/2020.acl-main.747.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[4] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in *Proc. 38th International Conference on Machine Learning*, vol. 139, 2021, pp. 8748–8763.

[5] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[6] ISO/IEC 25010:2023, *Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model*, 2nd ed., Nov. 2023.

[7] Object Management Group, *Unified Modeling Language (UML), Version 2.5.1*, Dec. 2017. Available: `https://www.omg.org/spec/UML/2.5.1/`. Accessed: 30-Jul-2026.

[8] Tesseract OCR, "Tesseract User Manual," 2026. Available: `https://tesseract-ocr.github.io/tessdoc/`. Accessed: 30-Jul-2026.

[9] FastAPI, "FastAPI in Containers—Docker," 2026. Available: `https://fastapi.tiangolo.com/deployment/docker/`. Accessed: 30-Jul-2026.

[10] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2026. Available: `https://www.postgresql.org/docs/current/`. Accessed: 30-Jul-2026.

[11] T. Tantau, "The TikZ and PGF Packages," 2026. Available: `https://ctan.org/pkg/pgf`. Accessed: 30-Jul-2026. (Diagramming tool used in this document.)